

2026



MATHÉMATIQUES ET INFORMATIQUE

Sciences

Université Paris Cité

Manuel d'installation



1

*Développement sous la
direction du groupe L2F2*

Direction : David JANISKEK

Manuel d'installation

Projet L2F2 – Déquantification

Identification du document		Éléments de vérification du document	
Référence	MI-L2F2	Validé par :	<i>Gaël MAHE</i>
Date	19/05/26	Validé le :	19/05/26
Auteurs	<i>KANGNI Dina, ACHAK Salim</i>	Soumis le :	19/05/26
Confidentialité :	Projet académique, réservé aux étudiants UFR Maths-Info de l'université Paris Descartes		

Élément d'authentification

	Maitrise d'ouvrage	<i>Gaël MAHE et David JANISZEK</i>	
	Maitrise d'œuvre	<i>Equipe L2F2</i>	
	Chef de projet	Dina KANGNI	

Sommaire

1. Introduction	6
2. Guide de lecture	6
3. Concepts de base	6
4. Installation du matériel	6
5. Paramétrage du système et résolution de problèmes	7
6. Procédure d'installation et de lancement de l'application	9
6.1. Installation des dépendances	9
Pour les utilisateurs Windows	9
Pour les utilisateurs Linux/MacOs	10
6.2. Lancement de l'application	10
Pour les utilisateurs Windows	10
Pour les utilisateurs Linux/MacOs	10
7. Autres informations	10
8. Glossaire	10
9. Références	11
10. Index	12

1. Introduction

Ce document décrit les procédures permettant d'installer et de lancer l'application de déquantification sur Windows et Linux. Les utilisateurs de MacOS peuvent se référer à celles portant sur Linux.

Cette application a été développée par KANGNI Dina, ACHAK Salim et BENALI Abdallah dans le cadre d'un projet universitaire organisé par l'UFR Mathématiques Informatique de l'Université Paris Cité.

Toute personne voulant utiliser ou tester l'application peut consulter ce manuel d'installation.

2. Guide de lecture

Ce cahier est destiné à tous les profils d'utilisateurs quel que soit leur niveau technique.

Des sections peuvent cependant être plus adaptées selon leurs besoins.

Les utilisateurs qui veulent simplement lancer l'application peuvent donc se référer aux sections 4 et 6 de ce cahier. Ceux qui utilisent Windows et rencontrent des problèmes d'installation de l'application peuvent aussi se référer à la section 5.

Les développeurs peuvent consulter les sections 5 et 7.

3. Concepts de base

Cette application de déquantification est développée avec le langage Julia. Elle nécessite plusieurs bibliothèques pour son fonctionnement dont GLMakie, une bibliothèque graphique, mentionnée dans la suite de ce document puisqu'elle est source de conflit lors de l'installation des dépendances.

4. Installation du matériel

L'utilisation et l'installation de l'application requièrent un ordinateur doté d'une carte graphique compatible avec au minimum OpenGL 3.3 et 4 Go de RAM. Julia 1.10.11 doit y être préalablement installé depuis son [site officiel](#).

L'utilisateur peut néanmoins installer d'autres versions de Julia plus récentes pour son utilisation personnelle, les scripts de lancement étant configurés pour lancer Julia 1.10.11 par défaut.

5. Paramétrage du système et résolution de problèmes

Au-delà de l'installation des dépendances, aucun paramétrage système particulier n'est nécessaire pour une utilisation de l'application.

Cependant, les utilisateurs Windows peuvent rencontrer des erreurs liées à la bibliothèque graphique GLMakie, notamment des fichiers DLL corrompus ou bloqués.

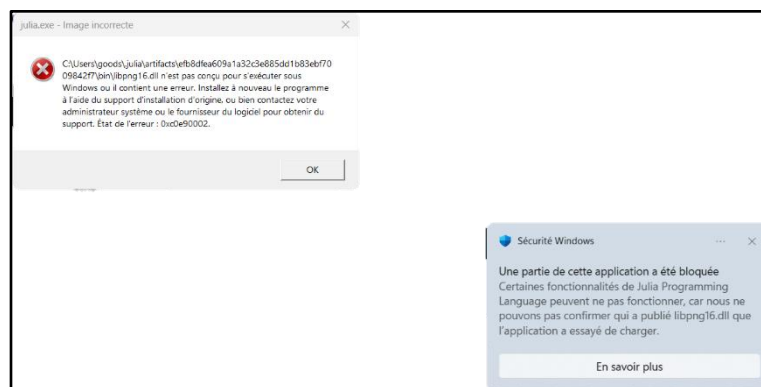


Figure 1 : Messages d'erreur indiquant le blocage des fichiers DLL lors du chargement de GLMakie.

Les fichiers DLL (*Dynamic Link Libraries*) sont des fichiers contenant du code partagé entre plusieurs programmes. Ils peuvent être bloqués par le système à cause d'un antivirus. Dans ce cas, la première solution possible est de désactiver temporairement l'antivirus ou d'ajouter le dossier *Julia* (généralement situé dans `%APPDATA%\julia`) à la liste des exclusions.

Les développeurs peuvent rencontrer des problèmes supplémentaires selon leur IDE. Les utilisateurs de VS Code (*Visual Studio Code*) en particulier peuvent observer des plantages de l'extension Julia accompagnés de messages d'erreur.

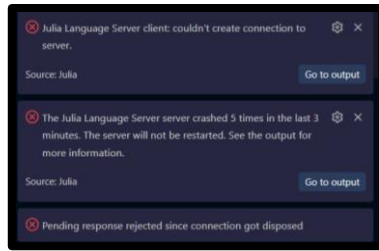


Figure 2 : Messages d'erreur indiquant le crash de l'extension Julia dans VS Code.

Dans ce cas, désinstaller puis réinstaller l'extension peut suffire à résoudre le problème.

Si des problèmes persistent, il est conseillé de privilégier un autre IDE.

Jupyter, par exemple, est un environnement interactif qui permet d'exécuter du code directement dans un navigateur.

Une autre solution consiste à travailler sous Linux. Les utilisateurs qui ne disposent pas d'une machine Linux native peuvent installer une distribution depuis une [machine virtuelle](#) et y déployer Julia en suivant les consignes dédiées aux utilisateurs Linux.

Les utilisateurs Windows peuvent également utiliser WSL (*Windows Subsystem for Linux*), qui permet d'exécuter un environnement Linux directement dans Windows sans machine virtuelle.

Il suffit pour cela de se rendre dans le PowerShell et d'y taper :

```
> wsl -install
```

Pour plus d'informations sur son installation, l'utilisateur peut se référer à [l'aide en ligne de Microsoft](#).

Il suffit ensuite d'installer Julia pour Linux. Il est également nécessaire d'installer [VcXsrv](#), un serveur d'affichage qui permet à GLMakie d'ouvrir sa fenêtre graphique.

La configuration du serveur se déroule via *XLaunch*, son interface de configuration.

Les paramètres suivants doivent être appliqués :

Sélection de **Multiple Windows**, définition du numéro d'affichage à **0**, sélection de **No client**, cochage de **Clipboard** et décochage **Wgl**.

6. Procédure d'installation et de lancement de l'application

6.1. Installation des dépendances

Les dépendances installées automatiquement sont GLMakie, GraphMakie, NativeFileDialog et ZipFile. Pour plus d'informations quant à leur rôle, le lecteur peut consulter le [glossaire](#).

Pour les utilisateurs Windows

L'utilisateur doit double-cliquer sur le fichier *install.bat* situé à la racine du projet. Ce script installe automatiquement toutes les dépendances nécessaires.

Il est également possible de l'exécuter depuis le terminal :

```
>cd chemin_projet  
>install.bat
```

Si Windows bloque l'exécution du fichier, l'utilisateur doit réaliser un clic droit sur *install.bat*, ouvrir *Propriétés*, puis dans l'onglet *Général*, section *Sécurité*, cocher *Débloquer*.

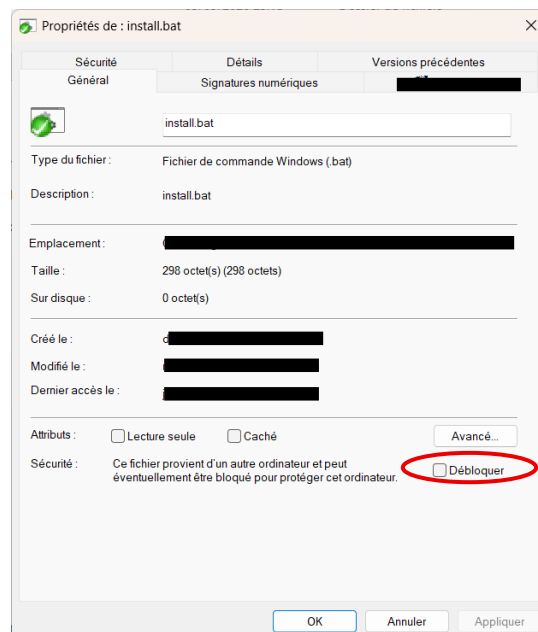


Figure 3 : Procédure de déblocage d'un fichier .bat.

En cas d'erreur, il peut désactiver temporairement l'antivirus, le temps de l'installation. Si le problème persiste, le lecteur est invité à consulter la section [5](#).

Pour les utilisateurs Linux/MacOs

L'utilisateur doit ouvrir un terminal et saisir :

```
>cd chemin_projet  
>chmod +x install.sh  
>./install.sh
```

Une fois les scripts de lancement exécutés, il n'est pas nécessaire de les relancer lors des prochaines utilisations de l'application.

6.2. Lancement de l'application

Pour les utilisateurs Windows

L'utilisateur peut double-cliquer sur le fichier run.bat ou ouvrir l'invite de commande et taper :

```
>run.bat
```

Pour les utilisateurs Linux/MacOs

L'utilisateur doit simplement ouvrir un terminal et taper :

```
>chmod +x run.sh  
>./run.sh
```

7. Autres informations

Pour mettre à jour l'application, il faut récupérer la nouvelle version depuis le dépôt SVN ou GitHub et relancer install.bat ou install.sh.

8. Glossaire

GLMakie : bibliothèque graphique Julia basée sur OpenGL.

Machine virtuelle : version logicielle d'un ordinateur physique.

Wsl (*Windows Subsystem for Linux*) : outil intégré à Windows permettant d'exécuter un environnement Linux directement depuis Windows, sans machine virtuelle.

VcXsrv : serveur permettant l'affichage des fenêtres graphiques d'un environnement Linux sous Windows via WSL. Il peut être paramétré via son interface XLaunch.

Fichiers DLL (*Dynamic Link Libraries*) : fichiers binaires Windows contenant du code compilé et des ressources partagés entre plusieurs programmes. Si une DLL est corrompue, le programme qui en dépend ne peut pas démarrer.

GLMakie : bibliothèque graphique Julia basée sur OpenGL, utilisée pour l'interface et les éléments interactifs de l'application.

GraphMakie : bibliothèque Julia reposant sur GLMakie, utilisée pour la visualisation de l'arbre servant à la déquantification dans l'application.

NativeFileDialog : bibliothèque Julia permettant d'ouvrir le sélecteur de fichiers natif du système d'exploitation.

ZipFile : bibliothèque Julia permettant de regrouper plusieurs fichiers dans une archive ZIP.

9. Références

al, J. B., Stefan Karpinski, Viral Shah, Alan Edelman, et. (s. d.). Installing julia. The Julia Programming Language. <https://julialang.org/downloads/index/>

GrantMeStrength. (s. d.). *Installer wsl*. <https://learn.microsoft.com/fr-fr/windows/wsl/install>

Oracle Corporation. (s. d.). *Downloads – Oracle VirtualBox*. VirtualBox. <https://www.virtualbox.org/wiki/Downloads>

Marchetti, M. (s. d.). *VcXsrv Windows X Server*. SourceForge. <https://sourceforge.net/projects/vcxsrv/>

10. Index

déquantification, 6
fichiers DLL, 7
GLMakie, 6, 7, 8, 9, 11
GraphMakie, 9, 11
IDE, 7, 8
Julia, 6, 7, 8, 11
Linux, 4, 6, 8, 10, 11
MacOs, 4, 6, 10, 11
NativeFileDialog, 9, 11
Windows, 4, 6, 7, 8, 9, 10, 12
WSL, 8
XLaunch, 8
ZipFile, 9, 11